Лабораторная работа №2

Q-обучение (Q-learning)

1. Общие сведения

Q-learning — это алгоритм обучения с подкреплением, который позволяет агенту оптимизировать свою стратегию действий в динамичной среде, стремясь максимизировать сумму будущих наград. Агент исследует среду, принимая решения, основанные на предыдущем опыте, а не на предварительной модели мира.

Принцип работы

Агент выбирает действия на основе данных **из Q-таблицы**, в которой изначально все значения равны нулю и таким образом на каждом шаге.

Q-таблица представляет собой матрицу, где строки обозначают возможные состояния среды, а столбцы — возможные действия агента в этих состояниях. Значения в Q-таблице отражают ожидаемую суммарную награду за выполнение действия в данном состоянии и следование оптимальной политике впоследствии.

В начале обучения Q-таблица **инициализируется нулями или случайными значениями**. Это представляет начальное предположение о том, насколько хороши различные действия в различных состояниях.

После каждого шага агента Q-значения в таблице обновляются с использованием уравнения Беллмана (о нем чуть позже).

Агент выбирает действия, основываясь на значениях из Q-таблицы, часто с использованием ε-жадной стратегии, которая балансирует между исследованием новых действий и эксплуатацией уже известных действий с максимальным Q-значением.

Допустим, есть простая среда с четырьмя возможными состояниями *S1, S2, S3, S4* и двумя возможными действиями *A1, A2* в каждом состоянии. Q-таблица для этой среды выглядела бы примерно так:

Состояние/действие	A1	A2
S1	0.5	0.2
S2	0.1	-0.4
S3	-0.2	0.0
S4	0.6	0.3

Значения в таблице указывают на ожидаемую суммарную награду за выбор действия A1 или A2 в соответствующем состоянии. Например, значение 0.5 в ячейке S1, A1 означает, что ожидаемая суммарная награда за выбор действия A1 в состоянии S1 составляет 0.5. С течением времени, по мере обучения агента, эти значения будут обновляться, отражая полученный опыт и уточняя оценки ожидаемых наград.

На каждом шаге агент обновляет Q-таблицу, используя **уравнение Беллмана**, учитывая полученные награды.

Уравнение Беллмана используется для расчета Q-значений, которые представляют собой оценки качества или полезности выполнения определенного действия из конкретного состояния. Эти Q-значения затем используются для определения оптимальной стратегии агента, направленной на максимизацию общей ожидаемой награды.

Уравнение Беллмана для Q-значений может быть выражено как:

$$Q(s, a) = r + \gamma \max_{a'} Q(s', a')$$

Где:

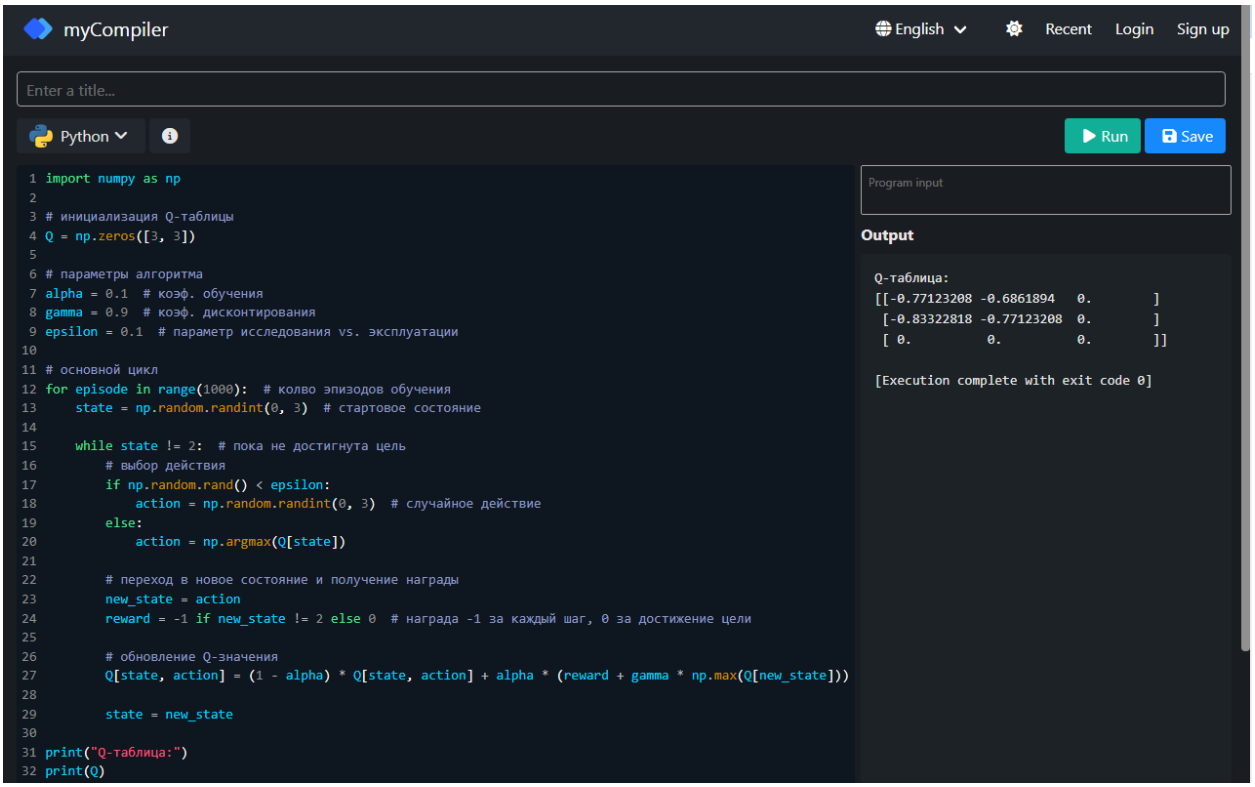
- *Q(s, a)* — ожидаемое Q-значение состояния и действия,
- *r* — немедленная награда, полученная после выполнения действия *a* из состояния *s*,
- *γ* — коэффициент дисконтирования, который представляет собой важность будущих наград (обычно значение между 0 и 1),
- A max и все то, что после него — максимальное Q-значение для всех возможных действий *a′* из следующего состояния *s′*.

Уравнение Беллмана по сути позволяет агенту учиться оценивать долгосрочные последствия своих действий, принимая во внимание текущую награду и ожидаемые будущие награды.

Таким образом можно рекурсивно раскрывать связь между Q-значениями текущих и будущих состояний.

Реализация алгоритма в Python

Первый пример с чистым нампаем, для задачи сетки 3x3:




```python
import numpy as np

# инициализация Q-таблицы
Q = np.zeros([3, 3])

# параметры алгоритма
alpha = 0.1  # коэф. обучения
gamma = 0.9  # коэф. дисконтирования
epsilon = 0.1  # параметр исследования vs. эксплуатации

# основной цикл
for episode in range(1000):  # колво эпизодов обучения
    state = np.random.randint(0, 3)  # стартовое состояние

    while state != 2:  # пока не достигнута цель
        # выбор действия
        if np.random.rand() < epsilon:
            action = np.random.randint(0, 3)  # случайное действие
        else:
            action = np.argmax(Q[state])

        # переход в новое состояние и получение награды
        new_state = action
        reward = -1 if new_state != 2 else 0  # награда -1 за каждый шаг, 0 за достижение цели

        # обновление Q-значения
        Q[state, action] = (1 - alpha) * Q[state, action] + alpha * (reward + gamma * np.max(Q[new_state]))

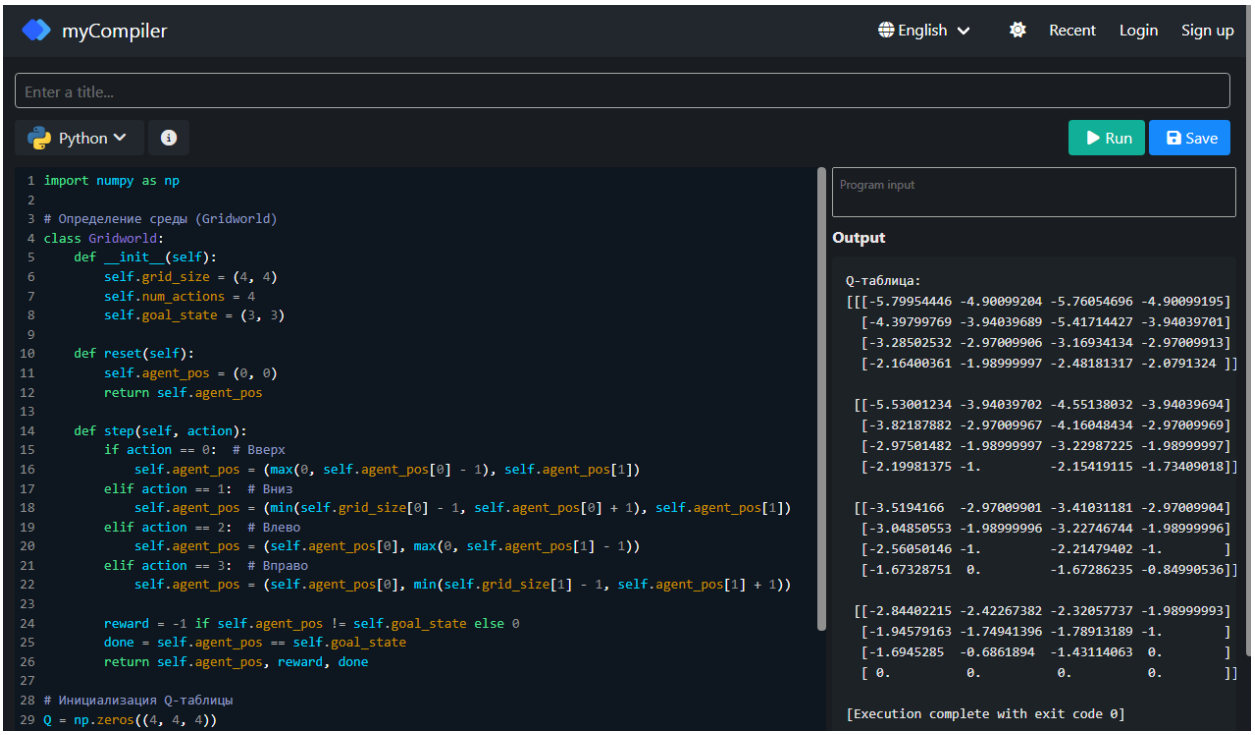
        state = new_state

print("Q-таблица:")
print(Q)
```

```
Q-таблица:
[[-0.77123208 -0.6861894   0.        ]
 [-0.83322818 -0.77123208  0.        ]
 [ 0.          0.          0.        ]]

[Execution complete with exit code 0]
```

Q-learning для простой среды Gridworld:




```python
import numpy as np

# Определение среды (Gridworld)
class Gridworld:
    def __init__(self):
        self.grid_size = (4, 4)
        self.num_actions = 4
        self.goal_state = (3, 3)

    def reset(self):
        self.agent_pos = (0, 0)
        return self.agent_pos

    def step(self, action):
        if action == 0:  # Вверх
            self.agent_pos = (max(0, self.agent_pos[0] - 1), self.agent_pos[1])
        elif action == 1:  # Вниз
            self.agent_pos = (min(self.grid_size[0] - 1, self.agent_pos[0] + 1), self.agent_pos[1])
        elif action == 2:  # Влево
            self.agent_pos = (self.agent_pos[0], max(0, self.agent_pos[1] - 1))
        elif action == 3:  # Вправо
            self.agent_pos = (self.agent_pos[0], min(self.grid_size[1] - 1, self.agent_pos[1] + 1))

        reward = -1 if self.agent_pos != self.goal_state else 0
        done = self.agent_pos == self.goal_state
        return self.agent_pos, reward, done

# Инициализация Q-таблицы
Q = np.zeros((4, 4, 4))
```

```
Q-таблица:
[[[-5.79954446 -4.90099204 -5.76054696 -4.90099195]
  [-4.39799769 -3.94039689 -5.41714427 -3.94039701]
  [-3.28502532 -2.97009906 -3.16934134 -2.97009913]
  [-2.16400361 -1.98999997 -2.48181317 -2.0791324 ]]

 [[-5.53001234 -3.94039702 -4.55138032 -3.94039694]
  [-3.82187882 -2.97009967 -4.16048434 -2.97009969]
  [-2.97501482 -1.98999997 -3.22987225 -1.98999997]
  [-2.19981375 -1.         -2.15419115 -1.73409018]]

 [[-3.5194166  -2.97009901 -3.41031181 -2.97009904]
  [-3.04850553 -1.98999996 -3.22746744 -1.98999996]
  [-2.56050146 -1.         -2.21479402 -1.        ]
  [-1.67328751  0.         -1.67286235 -0.84990536]]

 [[-2.84402215 -2.42267382 -2.32057737 -1.98999993]
  [-1.94579163 -1.74941396 -1.78913189 -1.        ]
  [-1.6945285  -0.6861894  -1.43114063  0.        ]
  [ 0.          0.          0.          0.        ]]

[Execution complete with exit code 0]
```

```python

# Параметры обучения
alpha = 0.1
gamma = 0.99
epsilon = 0.1
num_episodes = 1000

# Обучение
env = Gridworld()
for i in range(num_episodes):
    state = env.reset()
    done = False
    while not done:
        if np.random.rand() < epsilon:
            action = np.random.randint(0, env.num_actions)  # Случайное действие
        else:
            action = np.argmax(Q[state[0], state[1], :])  # Действие с максимальным Q-значением

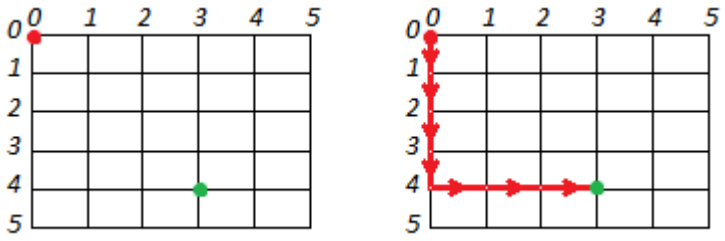
        next_state, reward, done = env.step(action)
        Q[state[0], state[1], action] = (1 - alpha) * Q[state[0], state[1], action] + alpha * (reward
        + gamma * np.max(Q[next_state[0], next_state[1], :]))

        state = next_state

print("Q-таблица:")
print(Q)
```

Обучение с подкреплением (Reinforcement Learning) — это область машинного обучения, в которой агент учится принимать решения, взаимодействуя с окружающей средой. Приведем простой пример, иллюстрирующий обучение с подкреплением с использованием Q-обучения (Q-learning).

В этом примере агент будет учиться перемещаться по простой сетке и получать награды за достижение цели:



myCompiler

Enter a title...

Python

```python
import numpy as np
import random

# Параметры среды
grid_size = 5
goal_state = (3, 4)
num_episodes = 1000
alpha = 0.1  # скорость обучения
gamma = 0.9  # коэффициент дисконтирования
epsilon = 0.1  # вероятность случайного действия

# Инициализация Q-таблицы
q_table = np.zeros((grid_size, grid_size, 4))  # 4 действия: вверх, вниз, влево, вправо

# Функция для выбора действия с использованием ε-жадной стратегии
def choose_action(state):
    if random.uniform(0, 1) < epsilon:
        return random.randint(0, 3)  # случайное действие
    else:
        return np.argmax(q_table[state])  # лучшее действие

# Функция для выполнения действия
def take_action(state, action):
    x, y = state
    if action == 0 and x > 0:  # вверх
        return (x - 1, y)
    elif action == 1 and x < grid_size - 1:  # вниз
        return (x + 1, y)
    elif action == 2 and y > 0:  # влево
        return (x, y - 1)
    elif action == 3 and y < grid_size - 1:  # вправо
        return (x, y + 1)
    return state  # если действие недопустимо, остаемся на месте

# Обучение агента
for episode in range(num_episodes):
    state = (0, 0)  # начальное состояние
    while state != goal_state:
        action = choose_action(state)
        next_state = take_action(state, action)

        # Награда: 1 за достижение цели, 0 в противном случае
        reward = 1 if next_state == goal_state else 0

        # Обновление Q-таблицы
        q_table[state][action] += alpha * (reward + gamma * np.max(q_table[next_state]) - q_table[state][action])

        state = next_state

# Проверка, как агент научился двигаться к цели
state = (0, 0)
path = [state]
while state != goal_state:
    action = np.argmax(q_table[state])
    state = take_action(state, action)
    path.append(state)

print("Найденный путь к цели:", path)
```

Output

```
Найденный путь к цели: [(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (1, 4), (2, 4), (3, 4)]

[Execution complete with exit code 0]
```

Пояснение к коду:

- **Q-таблица**: Хранит значения Q для каждого состояния и действия.
- **Выбор действия**: Используется ε-жадная стратегия для выбора действия, что позволяет агенту исследовать среду.
- **Обновление Q-таблицы**: Используется формула Q-обучения для обновления значений Q на основе полученной награды.
- **Обучение**: Агент обучается в течение заданного количества эпизодов, перемещаясь по сетке и получая награды.
- **Проверка**: После обучения агент показывает, как он достиг цели, следуя наилучшему пути.

Давайте разберем предложенный код для обучения с подкреплением, используя Q-обучение (Q-Learning), более подробно. Этот код иллюстрирует, как агент может учиться перемещаться по сетке и достигать цели, получая награды за свои действия.

Импорт библиотек

```
import numpy as np
import random
```

- **numpy**: Библиотека для работы с массивами и матрицами, а также для выполнения математических операций. В нашем коде она используется для создания и управления Q-таблицей.
- **random**: Библиотека для генерации случайных чисел. Она используется для реализации ε-жадной стратегии выбора действий.

Параметры среды

```
grid_size = 5
goal_state = (4, 4)
num_episodes = 1000
alpha = 0.1  # скорость обучения
gamma = 0.9  # коэффициент дисконтирования
epsilon = 0.1  # вероятность случайного действия
```

- **grid_size**: Размер сетки, в которой будет перемещаться агент. В данном случае это 5x5.
- **goal_state**: Координаты целевой ячейки, которую агент должен достичь (4, 4).
- **num_episodes**: Количество эпизодов, в течение которых агент будет обучаться. Каждый эпизод представляет собой отдельный запуск, в котором агент начинает с начального состояния и пытается достичь цели.
- **alpha**: Скорость обучения (learning rate), определяет, насколько сильно обновляется Q-значение при каждом шаге.
- **gamma**: Коэффициент дисконтирования, который определяет важность будущих наград по сравнению с текущими. Значение 0.9 говорит о том, что агент учитывает будущие награды, но с некоторым снижением.
- **epsilon**: Параметр для ε-жадной стратегии, который определяет вероятность случайного выбора действия, что позволяет агенту исследовать среду.

Инициализация Q-таблицы

```
q_table = np.zeros((grid_size, grid_size, 4))  # 4 действия: вверх, вниз, влево, вправо
```

- **q_table**: Это трехмерный массив, который хранит Q-значения для каждого состояния и действия. Размерность массива — `(grid_size, grid_size, 4)`, где `4` соответствует четырем возможным действиям (вверх, вниз, влево, вправо).

Функция для выбора действия

```
def choose_action(state):
    if random.uniform(0, 1) < epsilon:
        return random.randint(0, 3)  # случайное действие
    else:
        return np.argmax(q_table[state])  # лучшее действие
```

- **choose_action**: Функция, которая принимает текущее состояние и возвращает действие, которое агент должен выполнить.
- Если случайное число меньше `epsilon`, агент выбирает случайное действие (исследование). В противном случае он выбирает действие с максимальным Q-значением для текущего состояния (эксплуатация).

Функция для выполнения действия

```
def take_action(state, action):
    x, y = state
    if action == 0 and x > 0:  # вверх
        return (x - 1, y)
    elif action == 1 and x < grid_size - 1:  # вниз
        return (x + 1, y)
    elif action == 2 and y > 0:  # влево
        return (x, y - 1)
    elif action == 3 and y < grid_size - 1:  # вправо
        return (x, y + 1)
    return state  # если действие недопустимо, остаемся на месте
```

- **take_action**: Функция, которая принимает текущее состояние и действие, и возвращает новое состояние после выполнения действия.
- Каждое действие проверяет, допустимо ли оно (например, нельзя выйти за границы сетки). Если действие недопустимо, агент остается в том же состоянии.

Обучение агента

```
for episode in range(num_episodes):
    state = (0, 0)  # начальное состояние
    while state != goal_state:
        action = choose_action(state)
        next_state = take_action(state, action)

        # Награда: 1 за достижение цели, 0 в противном случае
        reward = 1 if next_state == goal_state else 0

        # Обновление Q-таблицы
        q_table[state][action] += alpha * (reward + gamma * np.max(q_table[next_state])
- q_table[state][action])

        state = next_state
```

- **Награда**: В этой части кода мы определяем награду, которую получает агент. Если агент достиг целевого состояния (`next_state == goal_state`), он получает награду `1` , в противном случае — `0` .

- **Обновление Q-таблицы**: Здесь происходит обновление Q-значения для текущего состояния и действия по формуле:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \quad r + \gamma \max_{a'} Q(s',a') - Q(s,a)$$

  - `s` — текущее состояние (state).
  - `a` — выбранное действие (action).
  - `r` — полученная награда (reward).
  - `s'` — следующее состояние (next_state).
  - `max_{a'} Q(s', a')` — максимальное Q-значение для следующего состояния, которое позволяет оценить наилучшее будущее действие.

  Таким образом, Q-значение обновляется, учитывая как текущую награду, так и ожидаемую будущую награду.

- **Переход в новое состояние**: После обновления Q-значения агент переходит в новое состояние (`state = next_state`), и цикл продолжается, пока агент не достигнет целевого состояния.

Проверка результатов

```
# Проверка, как агент научился двигаться к цели
state = (0, 0)
path = [state]
while state != goal_state:
    action = np.argmax(q_table[state])
    state = take_action(state, action)
    path.append(state)

print("Найденный путь к цели:", path)
```

- **Проверка**: После завершения обучения агент проверяет, как он научился достигать цели. Начинается с начального состояния `(0, 0)` и повторяет процесс выбора действий, используя максимальные Q-значения, пока не достигнет целевого состояния.

- **Сохранение пути**: Каждое новое состояние добавляется в список `path` , который хранит последовательность состояний, через которые агент прошел.

- **Вывод результата**: Наконец, программа выводит найденный путь к цели, показывая, как агент перемещался по сетке.

Этот код представляет собой простой, но эффективный пример Q-обучения. Он демонстрирует основные концепции, такие как:

- **Q-таблица**: Хранит значения Q для различных состояний и действий.
- **ε-жадная стратегия**: Балансирует между исследованием и эксплуатацией.
- **Обновление Q-значений**: Позволяет агенту адаптироваться к окружающей среде на основе полученных наград.

Содержание работы.

1. **Какой алгоритм называют Q-learning?**
2. **Каков принцип работы алгоритма Q-обучения?**
3. **Как выглядит уравнение Беллмана для Q-значений?**
4. **Реализуйте алгоритм вывода Q-таблицы на Python для задачи сетки 3x3.**
5. **Реализуйте алгоритм вывода Q-таблицы на Python для простой среды Gridworld.**
6. **Реализуйте на Python алгоритм Q-обучения, где агент учиться перемещаться по простой прямоугольной сетке и получает награды за достижение цели. Осуществите вывод найденного пути к цели.**